

Facultad Regional San Nicolas

Tecnicatura Universitaria en Programación

Materia: Arquitectura y Sistemas Operativos

Trabajo Práctico Integrador

Proyecto: Virtualización.

Integrantes:

Sebastian Cerizza - Legajo: [104492]

Maximiliano Javier Castillo - Legajo: [103062]

Fecha de Entrega: 21 de Junio de 2026

ÍNDICE

1. Introducción	3
2. Marco Teórico	3
3. Caso Práctico	6
4. Metodología Utilizada	16
5. Resultados Obtenidos	17
6. Conclusiones	18
7. Bibliografía	18
8. Anexo	19

1. INTRODUCCIÓN:

Sobre el tema a realizar el TPI hemos elegido el trabajo de virtualización ya que nos pareció un tema fundamental para el desarrollo de las tareas de un programador. Esto se debe a que al utilizar entornos virtuales nos permite desarrollar y ejecutar software sin depender del hardware.

Para nuestro desarrollo como técnicos tiene una importancia clave ya que nos permite prepararnos para una metodología de trabajo que va a ser fundamental en nuestras carreras. Ya que por la flexibilidad y seguridad que permite esta modalidad de trabajo suponemos que será clave en nuestro futuro

Nuestros objetivos para este trabajo es comprender el funcionamiento de una máquina virtual (en especial dentro de un entorno de hypervisores tipo 2). Poder realizar la instalación y ejecución de una máquina virtual en nuestro sistema operativo. Y, por último, poder utilizar esa máquina virtual para el desarrollo de una aplicación a través de un lenguaje de programación instalado en la misma (python).

2. MARCO TEÓRICO:

La virtualización consiste en la ejecución de versiones virtuales de recursos informáticos que no dependen del hardware.

Esto genera múltiples beneficios en cuanto al desarrollo de nuestros programas entre los que podemos listar:

-Eficiencia: Maximiza el uso de los recursos, reduciendo el desperdicio y mejorando la utilización del hardware.

-Flexibilidad: Facilita el desarrollo, pruebas y despliegue de software en entornos aislados.

-Escalabilidad: Permite adaptar fácilmente los recursos a las necesidades cambiantes, sin la necesidad de comprar hardware adicional.

-Ahorro de Costos: Reduce la necesidad de adquirir múltiples servidores físicos, lo que genera ahorros en costos de hardware, energía y mantenimiento.

Para poder generar estas máquinas virtuales necesitamos un hypervisor. Este mismo consiste en un software que crea y gestiona las máquinas virtuales (VMs). También actúa como intermediario entre el hardware físico y los SO de las VMs.

Existen 2 tipo de hypervisores:

Tipo 1: En este caso el software del hypervisor está montado directamente sobre el hardware del equipo. Ejemplos: VMware ESXi, Hyper-V, Xen.

Tipo 2: Aquí el hypervisor se monta como un programa dentro del SO anfitrión. Ejemplos: VirtualBox, VMware Workstation.

Diferencias entre hypervisores tipo 1 y tipo 2:

Hypervisores tipo 1	Hypervisores tipo 2
Mayor rendimiento	Menor rendimiento por capa adicional (SO anfitrión)
Eficiencia en la gestión de recursos	Dependencia de la asignación de recursos del SO anfitrión
Compleja instalación y configuración	Fácil instalación y uso
Ideal para producción	Ideal para pruebas y desarrollo
Requiere hardware dedicado	Se monta sobre hardware disponible

Para nuestro trabajo nos vamos a enfocar en la virtualización tipo 2, así que vamos a ver cómo accede la misma a los recursos.

Características:

- Se ejecuta sobre un sistema operativo anfitrión.
- Utiliza las APIs y servicios del anfitrión para gestionar recursos.
- Proporciona un entorno virtualizado a las máquinas virtuales.

Virtualización del CPU:

Este recurso permite el acceso a múltiples sistemas operativos al mismo procesador sin que se interfieran entre sí. Para esto utiliza 2 métodos principales:

1- Traducción Binaria: El hipervisor intercepta instrucciones privilegiadas del sistema operativo invitado (Guest OS) y las traduce a instrucciones seguras.

2- Soporte de Virtualización por Hardware (Intel VT-x y AMD-V): Esta tecnología permite dividir al procesador en contextos separados para que cada VMs pueda acceder al mismo sin necesidad de realizar una traducción.

Virtualización de la memoria:

Aquí veremos los mecanismos que utiliza el hipervisor para hacer que las VMs no accedan al mismo lugar de la memoria:

1. Tablas de Páginas Sombreadas (Shadow Page Tables): Consiste en mapear las direcciones virtuales a las que accede cada VMs para asignarle una dirección física real del anfitrión.

2. Second Level Address Translation (SLAT): Es una extensión en la cual permite hacer la traducción de memoria directamente al procesador, quitándole esa carga al hipervisor y mejorando el rendimiento.

Virtualización del Almacenamiento:

Esto permite generar contenedores en la memoria física del hardware que consiste en discos virtuales a los cuales acceden las VMs como si fueran discos duros. Esto como ventaja permite redimensionar, clonar o crear snapshot fácilmente sin necesidad real de modificar el hardware (siempre que este permita el cambio)

Virtualización de la Red:

El hipervisor permite crear una red virtual para poder conectar las VMs entre sí y con la red exterior. Puede usar distintos modos como NAT en el cual las VMs comparten la IP del anfitrión, lo que simplifica el acceso a internet. BRIDGED NETWORKING, aquí cada VM

recibe su propia IP en la red física, ideal para entornos de prueba avanzados. RED INTERNA, sirve exclusivamente para comunicación entre VMs dentro del mismo hypervisor

Virtualización de Dispositivos:

Esto permite que las VMs interactúen con un hardware real o emulado. Existen 2 tipos:

Dispositivos Emulados: Un ejemplo sería una GPU virtual lo que permite un renderizado básico para ver el escritorio de la misma pero con un rendimiento limitado.

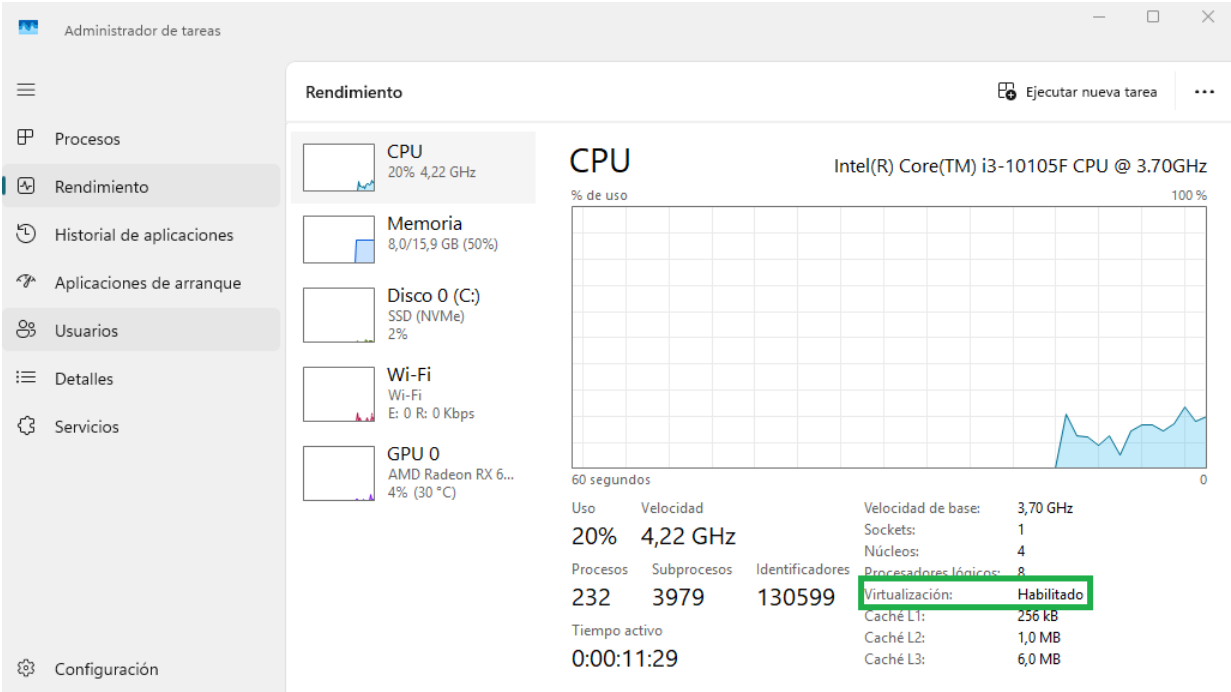
Passthrough de Dispositivos: Aquí las VMs acceden directamente al hardware real para tener un mayor rendimiento. En este caso se podría hablar de una GPU real la cual le permite a la VM realizar un renderizado 3D.

Documentación utilizada:

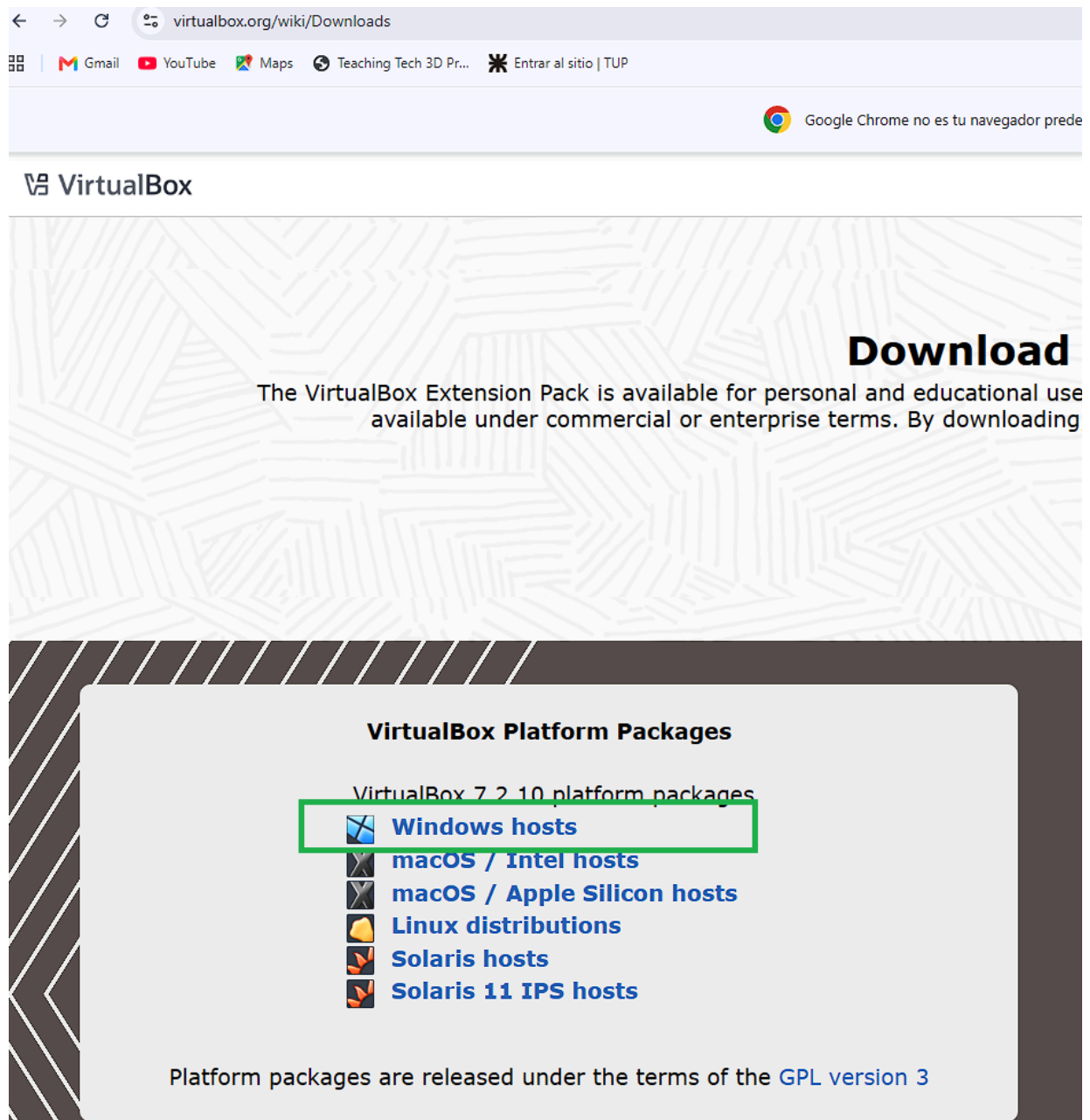
- https://tup.sied.utn.edu.ar/pluginfile.php/26094/mod_label/intro/Virtualizacion%20de%20recursos%20por%20un%20hipervisor%20tipo%202.pdf
- https://tup.sied.utn.edu.ar/pluginfile.php/26094/mod_label/intro/Virtualizacion-de-Recur-sos-por-un-Hypervisor-Tipo-2.pdf
- https://tup.sied.utn.edu.ar/pluginfile.php/26094/mod_label/intro/Virtualizacion-Concepto-s-y-Beneficios.pdf

3. CASO PRÁCTICO:

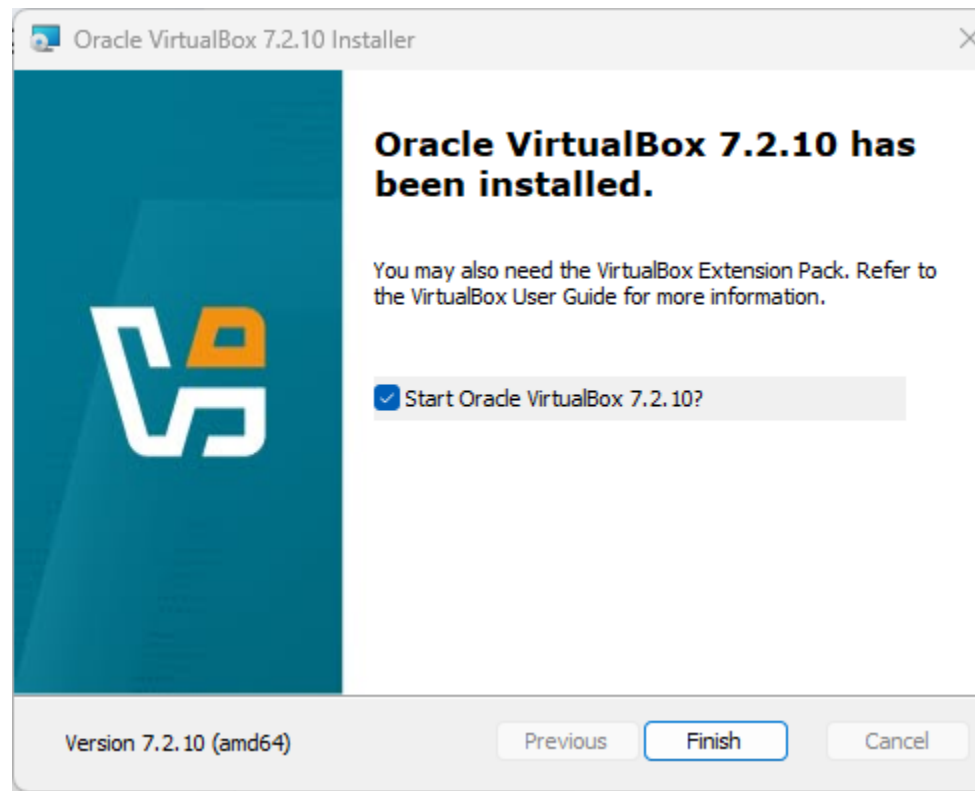
En este caso vamos a realizar un programa en python que le permita al usuario obtener el promedio de res valores ingresados, todo ejecutado desde un entorno virtual para poder ver cómo se realizaría en la vida real para desarrollar un software. Para realizar esto primero vamos a descargar e instalar un hypervisor (VirtualBox) y la imagen de un SO (Ubuntu) el cual va a ser el SO que instalaremos en nuestra VM. Luego, a través de la terminal generamos el archivo python en cual desarrollaremos el programa. Todo esto será subido a un repositorio de github.



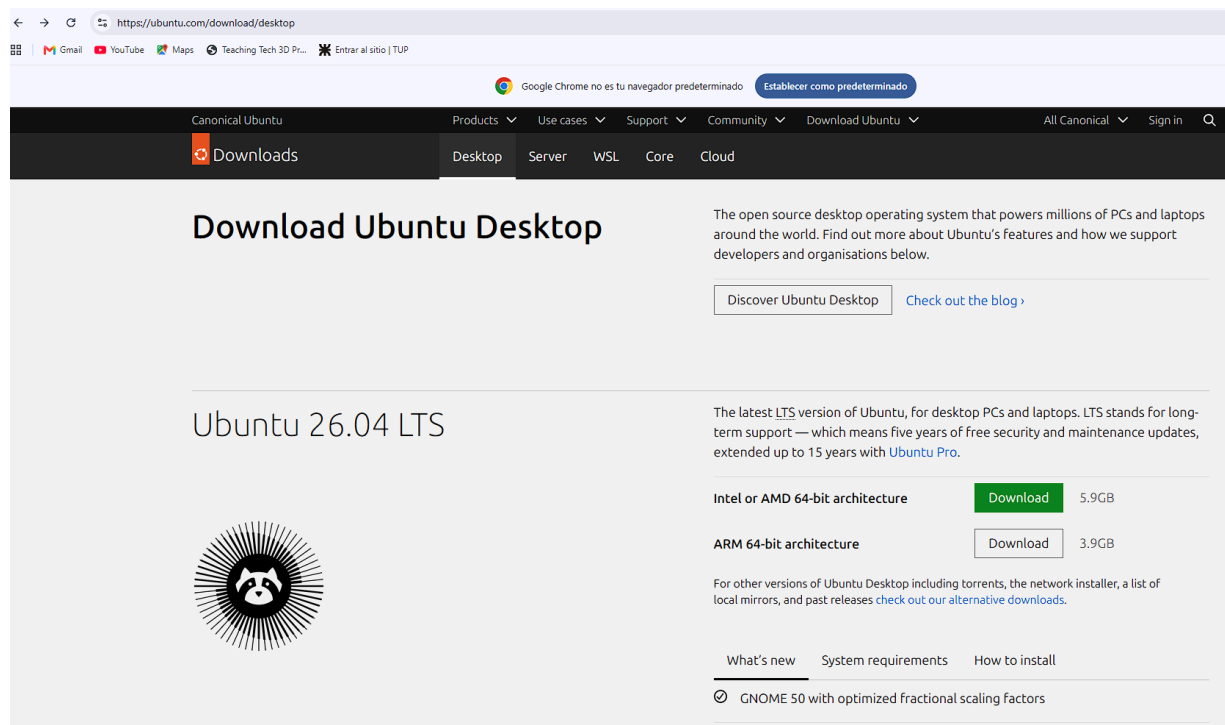
- Validación de virtualización habilitada desde el administrador de tareas.



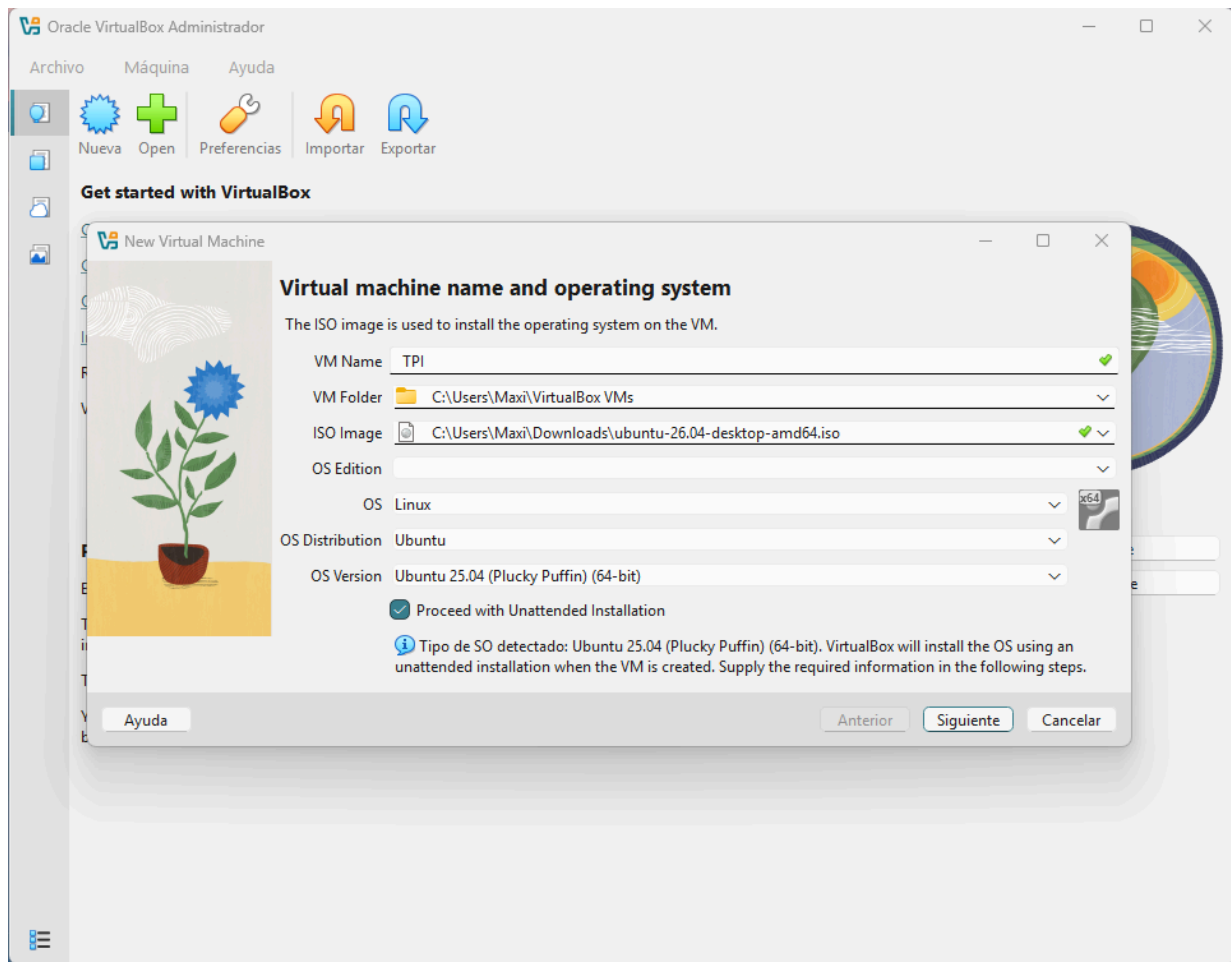
- Desde la url <https://www.virtualbox.org/wiki/Downloads> descargamos la versión para windows.



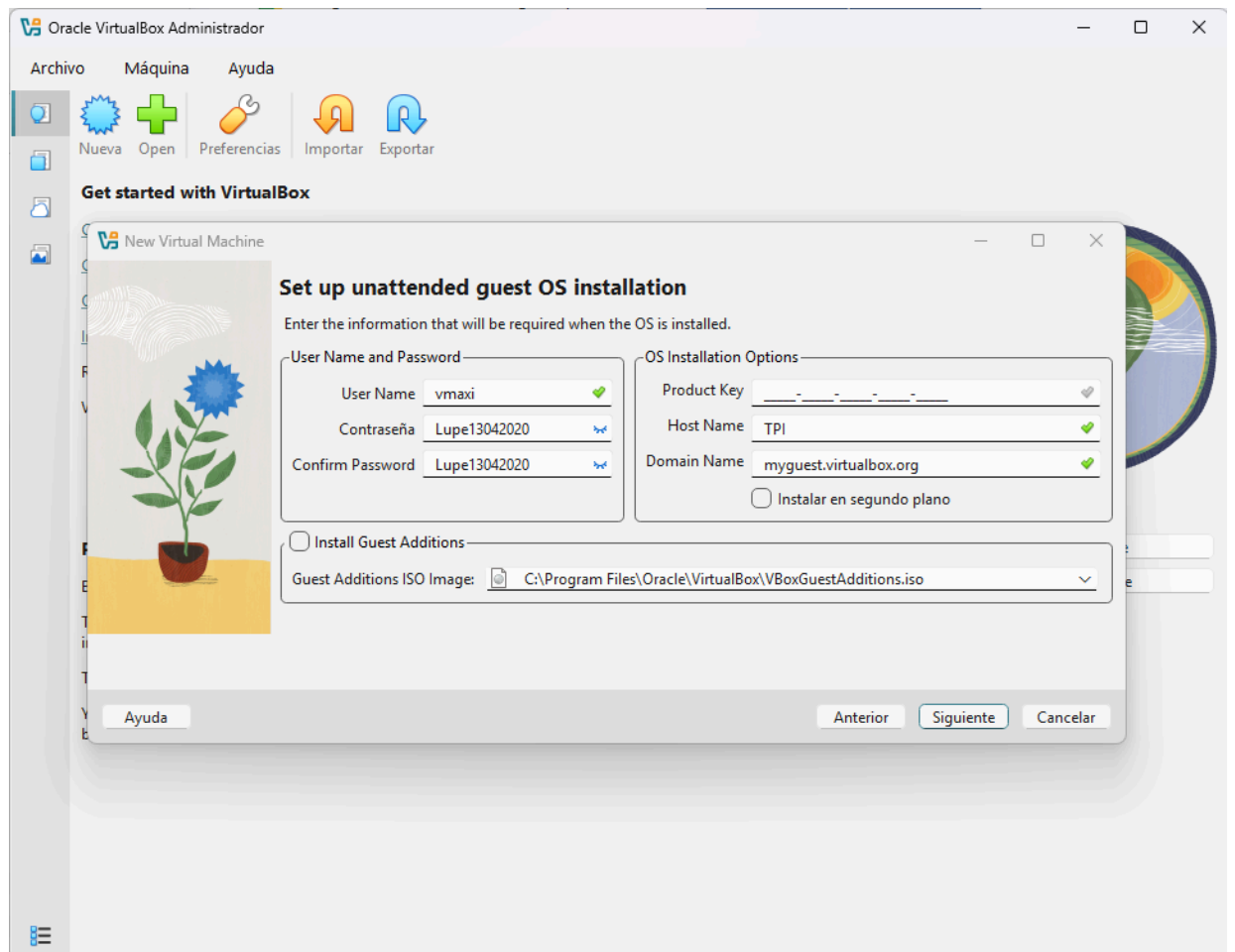
- Luego ejecutamos el archivo.exe e instalamos virtualbox



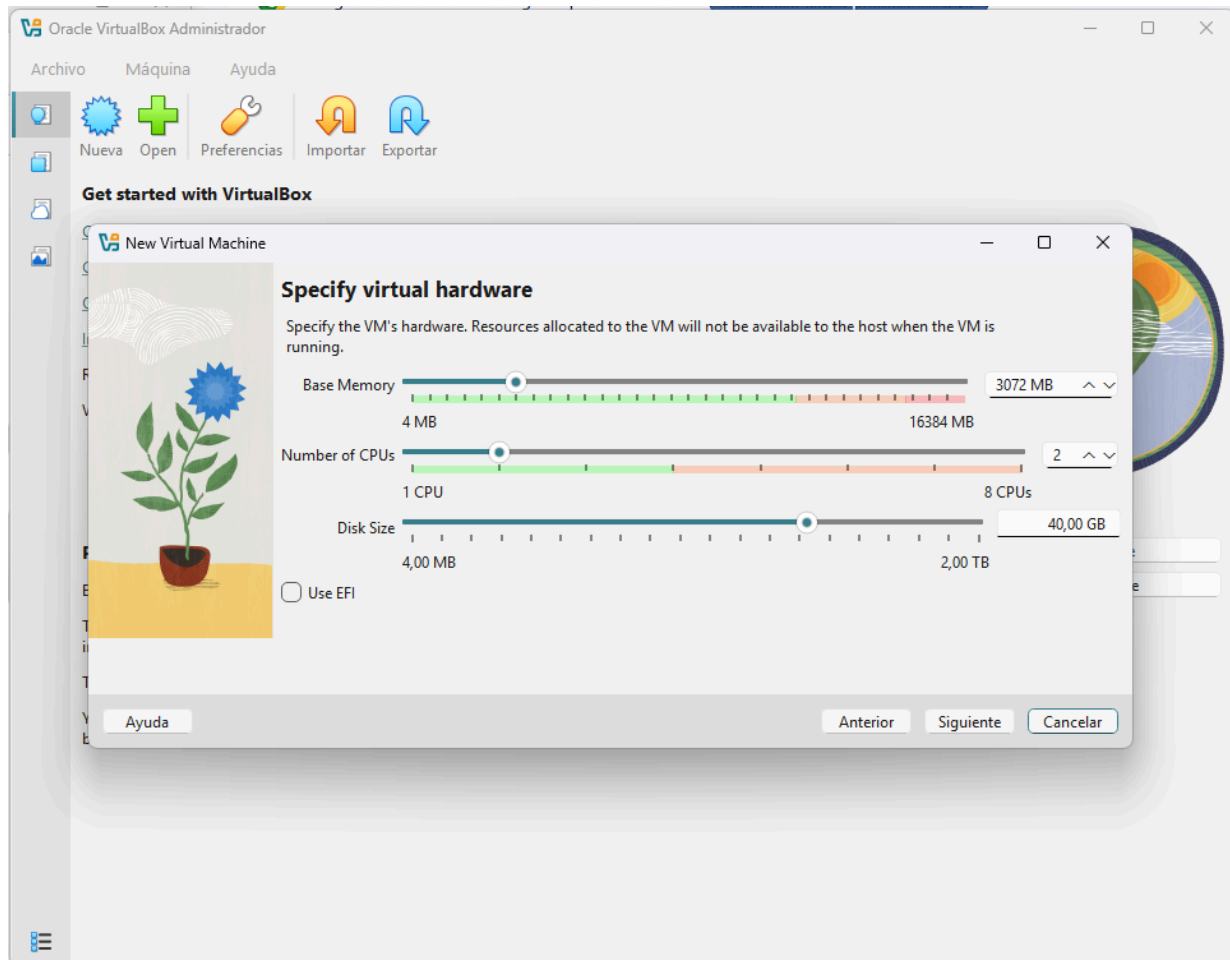
- Descargamos la imagen de Linux ubuntu desde <https://ubuntu.com/download/desktop>



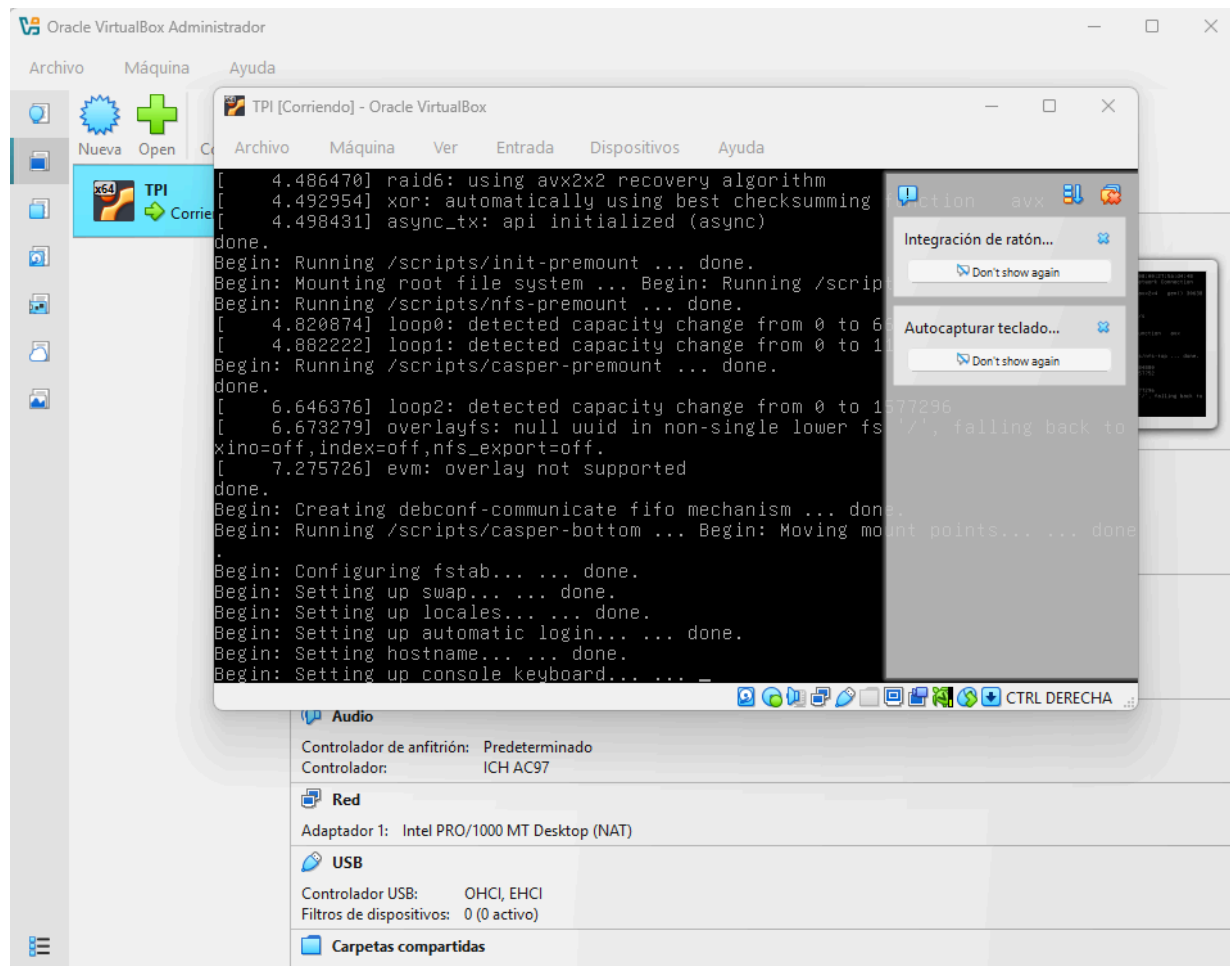
- Iniciamos VirtualBox y creamos una nueva máquina virtual, con la imagen de ubuntu descargada previamente.



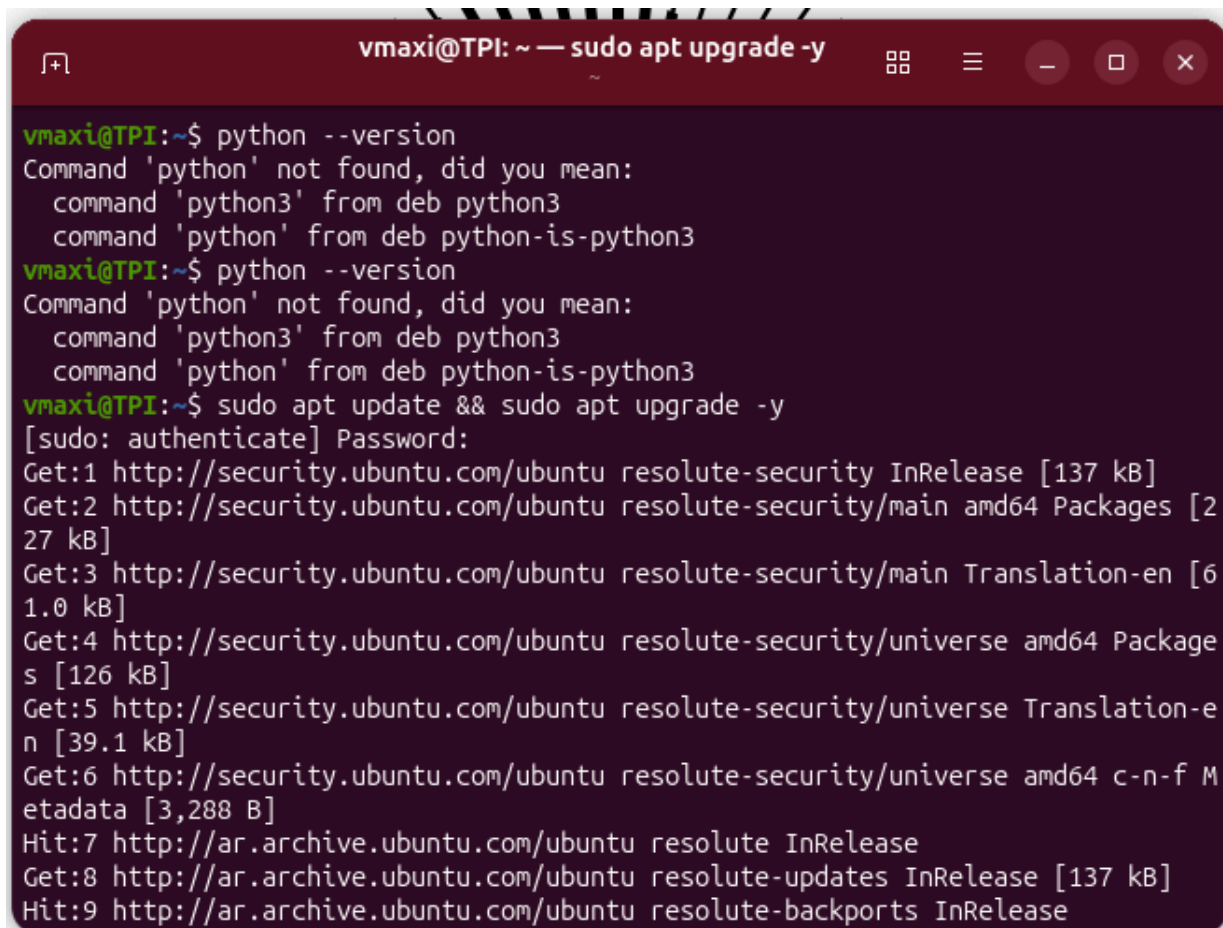
- configuramos usuario y password para poder ingresar



- Definimos las especificaciones técnicas memoria, cpu, disco en base a lo que dimensionamos como requerimiento básico.

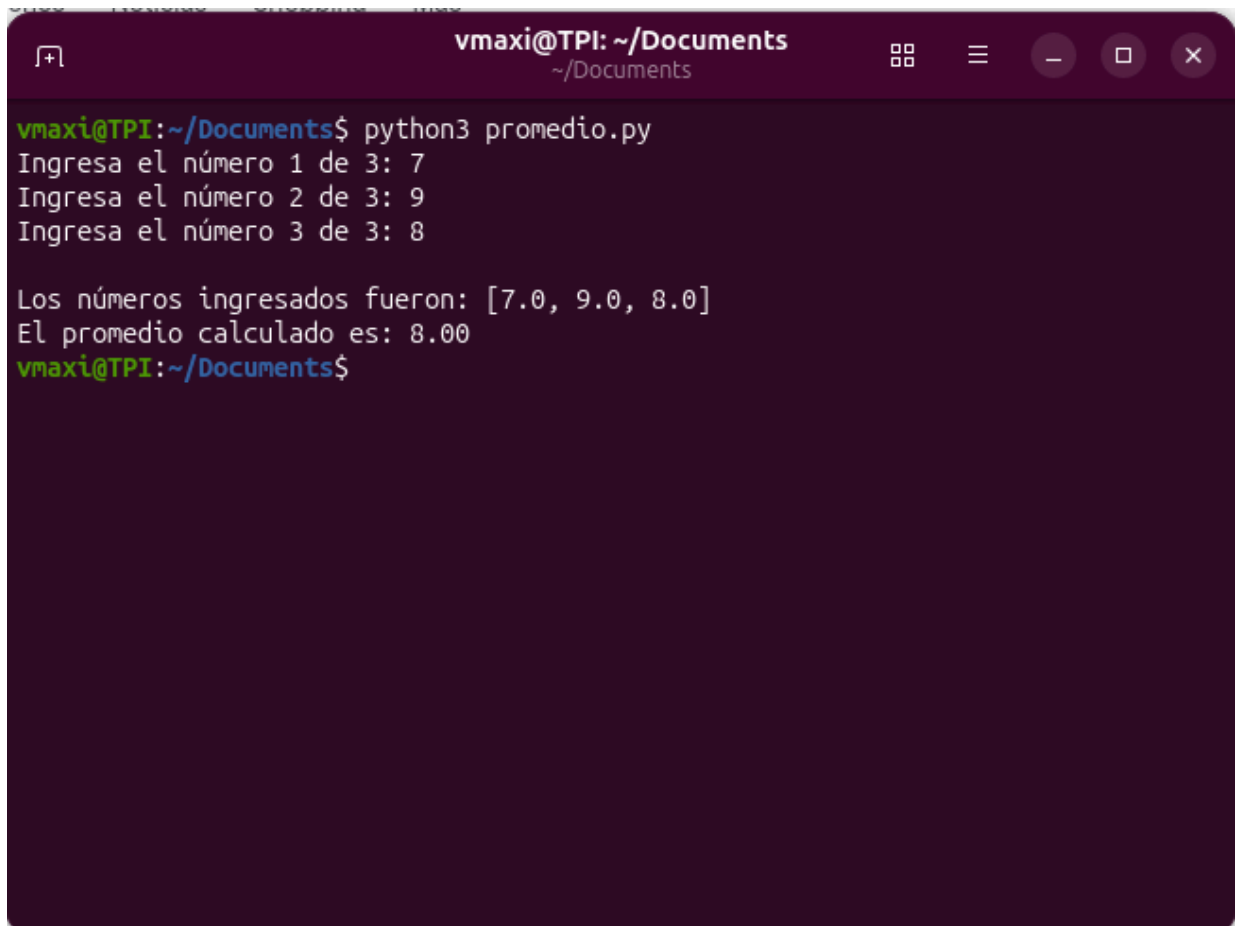


- Una vez iniciada la instalación de ubuntu, aguardamos a que termine.



```
vmaxi@TPI: ~ — sudo apt upgrade -y
vmaxi@TPI:~$ python --version
Command 'python' not found, did you mean:
  command 'python3' from deb python3
  command 'python' from deb python-is-python3
vmaxi@TPI:~$ python --version
Command 'python' not found, did you mean:
  command 'python3' from deb python3
  command 'python' from deb python-is-python3
vmaxi@TPI:~$ sudo apt update && sudo apt upgrade -y
[sudo: authenticate] Password:
Get:1 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Get:2 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [227 kB]
Get:3 http://security.ubuntu.com/ubuntu resolute-security/main Translation-en [61.0 kB]
Get:4 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Packages [126 kB]
Get:5 http://security.ubuntu.com/ubuntu resolute-security/universe Translation-en [39.1 kB]
Get:6 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 c-n-f Metadata [3,288 B]
Hit:7 http://ar.archive.ubuntu.com/ubuntu resolute InRelease
Get:8 http://ar.archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Hit:9 http://ar.archive.ubuntu.com/ubuntu resolute-backports InRelease
```

- Luego instalamos python y pip.

A terminal window titled 'vmaxi@TPI: ~/Documents' with standard window controls. The terminal shows the execution of a Python script. The user enters 'python3 promedio.py'. The script prompts for three numbers: 'Ingresa el número 1 de 3: 7', 'Ingresa el número 2 de 3: 9', and 'Ingresa el número 3 de 3: 8'. It then outputs 'Los números ingresados fueron: [7.0, 9.0, 8.0]' and 'El promedio calculado es: 8.00'. The prompt returns to 'vmaxi@TPI:~/Documents\$'.

```
vmaxi@TPI:~/Documents$ python3 promedio.py
Ingresa el número 1 de 3: 7
Ingresa el número 2 de 3: 9
Ingresa el número 3 de 3: 8

Los números ingresados fueron: [7.0, 9.0, 8.0]
El promedio calculado es: 8.00
vmaxi@TPI:~/Documents$
```

- Probamos la instalación de python con la aplicación nuestro repositorio.

4. METODOLOGÍA UTILIZADA

El desarrollo del proyecto se llevó a cabo mediante un enfoque secuencial y estructurado, abarcando desde la preparación del entorno de virtualización hasta la ejecución de la aplicación de prueba. Las etapas se detallan a continuación:

- **Aprovisionamiento del Hipervisor:** Se procedió con la descarga del instalador de **VirtualBox** desde su sitio web oficial, garantizando la integridad y seguridad del software. Posteriormente, se realizó la instalación en el sistema operativo Host (**Windows**)

- **Obtención del Sistema Operativo Invitado:** Se descargó la imagen ISO oficial de **Ubuntu LTS** (Long Term Support) para asegurar un entorno de ejecución estable, documentado y con soporte extendido.
- **Dimensionamiento de Recursos con Asistencia de IA:** Con el objetivo de optimizar el rendimiento del sistema y evitar el estrangulamiento (*throttling*) de la máquina Host, se utilizó **Gemini** como herramienta de asistencia para la estimación de recursos. En base a las especificaciones del hardware físico, se determinó la asignación óptima de memoria RAM, núcleos de CPU y almacenamiento para la máquina virtual (VM).
- **Despliegue y Configuración del Entorno de Desarrollo:** Una vez inicializado el sistema operativo invitado, se utilizó la interfaz de línea de comandos (CLI) para actualizar los repositorios e instalar **Python** junto con su gestor de paquetes **pip**. Esto proveyó el *runtime* necesario para la ejecución de scripts.
- **Clonación y Ejecución de la Aplicación:** Finalmente, se utilizó **Git** (o descarga directa) para obtener desde un repositorio de GitHub el código fuente de la aplicación `promedio.py`, procediendo a su ejecución en el entorno aislado de la VM.

5. RESULTADOS OBTENIDOS

Los indicadores de éxito del trabajo práctico se consolidaron en los siguientes puntos:

- **Estabilidad del Entorno Virtual:** El sistema operativo invitado (Ubuntu) se ejecutó de manera fluida y correcta. No se presentaron fallos ni degradación crítica en el rendimiento del sistema Host, validando que el dimensionamiento previo de recursos fue adecuado.
- **Correcto Funcionamiento Python :** La instalación de Python y pip por terminal fue exitosa. La aplicación `promedio.py` se ejecutó sin lanzar excepciones de dependencias o de entorno, procesando los datos de manera correcta.
- **Comprensión de la Arquitectura Host-Guest:** Se logró un entendimiento práctico de la interacción entre el sistema Host y el Guest. Se validaron conceptos clave como el aislamiento de procesos

6. CONCLUSIONES

La virtualización se consolida como una herramienta indispensable en el ámbito del desarrollo de software, ya que permite generar **entornos de ejecución aislados y controlados**. Esto facilita la realización de pruebas de compatibilidad sobre diferentes sistemas operativos, con la ventaja crítica de que el sistema Guest no comparte el *kernel* con el sistema Host, garantizando que cualquier fallo catastrófico en la VM no afecte la estabilidad de la máquina física.

No obstante, la experiencia de este laboratorio demuestra que la virtualización requiere una planificación rigurosa de los recursos de hardware. Al momento de dimensionar una máquina virtual, es mandatorio evaluar tanto los requisitos mínimos del sistema operativo elegido como la carga de trabajo de las aplicaciones que se van a ejecutar en ella. Validar la disponibilidad de hardware real (RAM y CPU) antes del despliegue es un paso mandatorio para asegurar la viabilidad de cualquier proyecto de infraestructura virtual.

7. BIBLIOGRAFÍA

Apuntes de la catedra AySO UTN-FRSN

- https://tup.sied.utn.edu.ar/pluginfile.php/26094/mod_label/intro/Virtualizacion%20de%20recursos%20por%20un%20hipervisor%20tipo%202.pdf
- https://tup.sied.utn.edu.ar/pluginfile.php/26094/mod_label/intro/Virtualizacion-de-Recur-sos-por-un-Hypervisor-Tipo-2.pdf
- https://tup.sied.utn.edu.ar/pluginfile.php/26094/mod_label/intro/Virtualizacion-Concepto-s-y-Beneficios.pdf

Link a programas utilizados:

- <https://www.virtualbox.org/wiki/Downloads>
- <https://ubuntu.com/#download-ubuntu>

8. ANEXO:

Link a video youtube:

<https://youtu.be/cobix4bD-QQ>